

4. Binary Search with Big-O Analysis

Aim

To write a Python program to implement **Binary Search** and analyze its **Big-O time complexity**.

Algorithm

1. Start.
2. Take a **sorted array**.
3. Set $low = 0$ and $high = n - 1$.
4. Find the middle element.
5. If the middle element equals the target, return its position.
6. If the target is smaller, search the left half.
7. If the target is larger, search the right half.
8. Repeat until the element is found or the search range becomes empty.
9. Stop.

Source Code

```
def binary_search(arr, target):
```

```
    low = 0
```

```
    high = len(arr) - 1
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```

```
        if arr[mid] == target:
```

```
            return mid
```

```
        elif arr[mid] < target:
```

```
            low = mid + 1
```

```
    else:
```

```
        high = mid - 1

    return -1

arr = [10, 20, 30, 40, 50, 60, 70]

target = 40

result = binary_search(arr, target)

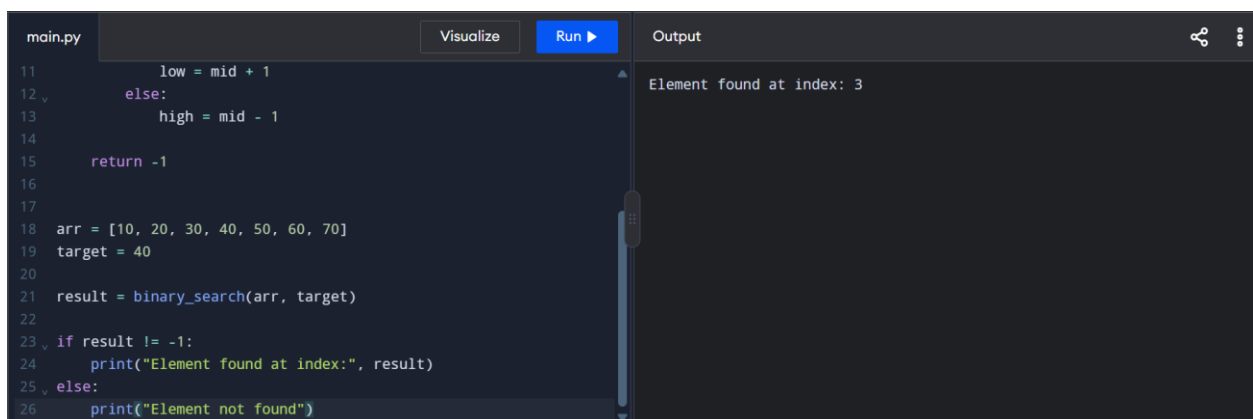
if result != -1:

    print("Element found at index:", result)

else:

    print("Element not found")
```

Output

A screenshot of a code editor interface. The left pane shows a Python file named 'main.py' with the following code:

```
11     low = mid + 1
12     else:
13         high = mid - 1
14
15     return -1
16
17
18 arr = [10, 20, 30, 40, 50, 60, 70]
19 target = 40
20
21 result = binary_search(arr, target)
22
23 if result != -1:
24     print("Element found at index:", result)
25 else:
26     print("Element not found")
```

 The right pane, titled 'Output', displays the result of running the code: 'Element found at index: 3'. Above the code editor are buttons for 'Visualize' and 'Run'. The 'Run' button is highlighted in blue.

Big-O Analysis

- **Best Case:** $O(1)$
- **Average Case:** $O(\log n)$
- **Worst Case:** $O(\log n)$
- **Space Complexity:** $O(1)$

Result

Thus, the Binary Search program was successfully implemented. The worst-case time complexity is $O(\log n)$ and the space complexity is $O(1)$.